

Correction — PF2 Partiel 5

Programmation Fonctionnelle 2 — juin 2026 — Pierre Rousselin
Université Sorbonne Paris Nord

Exercice 1 — Preuves de logique des propositions (12 pts)

1. Lemma ex5_1 : $(A \vee B) \wedge \sim A \rightarrow B$ (4 pts)

Terme de preuve :

```
exact (fun h =>
  match h with conj hAB hna =>
    match hAB with
    | or_introl hA => False_ind B (hna hA)
    | or_intror hB => hB
  end
end).
```

Avec tactiques :

```
Proof.
  intros [[hA | hB] hna].
  - exfalso. apply hna. exact hA.
  - exact hB.
Qed.
```

Arbre de dérivation :

$\frac{h : (A \vee B) \wedge \neg A}{\vdash A \vee B} \quad (\wedge E - g)$	$\frac{h : (A \vee B) \wedge \neg A}{\vdash \neg A} \quad (\wedge E - d)$
$\frac{[hA : A] \quad \vdash \neg A \text{ (depuis h)}}{\vdash \text{False}} \quad (\rightarrow E)$	$\frac{[hB : B]}{\vdash B} \quad (Ax)$
$\frac{\vdash \text{False}}{\vdash B} \quad (\text{exfalso})$	
$\frac{\vdash B}{\vdash (A \vee B) \wedge \neg A \rightarrow B} \quad (\vee E \text{ sur } A \vee B)$	
$\frac{h : (A \vee B) \wedge \neg A \vdash B}{\vdash (A \vee B) \wedge \neg A \rightarrow B} \quad (\rightarrow I \text{ sur } h)$	

2. Lemma ex5_2 : $\sim A \vee \sim B \rightarrow \sim (A \wedge B)$ (4 pts)

Terme de preuve :

```

exact (fun h hAB =>
  match h with
  | or_introl hna => hna (match hAB with conj hA _ => hA end)
  | or_intror hnb => hnb (match hAB with conj _ hB => hB end)
end).

```

Avec tactiques :

```

Proof.
  intros [hna | hnb] hAB.
  - apply hna. destruct hAB as [hA _]. exact hA.
  - apply hnb. destruct hAB as [_ hB]. exact hB.
Qed.

```

Arbre de dérivation :

$\frac{[hAB:A \wedge B]}{\text{---}} (\wedge E-g)$ $\vdash A$ $hna:\neg A$ $\text{---} (\rightarrow E)$ $\vdash \text{False}$ $\text{---} (\rightarrow I \text{ sur } hAB)$ $hna \vdash \neg(A \wedge B)$	$\frac{[hAB:A \wedge B]}{\text{---}} (\wedge E-d)$ $\vdash B$ $hnb:\neg B$ $\text{---} (\rightarrow E)$ $\vdash \text{False}$ $\text{---} (\rightarrow I \text{ sur } hAB)$ $hnb \vdash \neg(A \wedge B)$
$\text{---} (\vee E \text{ sur } h:\neg A \vee \neg B)$	
$h:\neg A \vee \neg B \vdash \neg(A \wedge B)$	
$\text{---} (\rightarrow I \text{ sur } h)$	
$\vdash (\neg A \vee \neg B) \rightarrow \neg(A \wedge B)$	

3. Lemma ex5_3 : (A -> B \ / C) -> ~ C -> A -> B (4 pts)

Terme de preuve :

```

exact (fun h hnc hA =>
  match h hA with
  | or_introl hB => hB
  | or_intror hC => False_ind B (hnc hC)
end).

```

Avec tactiques :

```

Proof.
  intros h hnc hA.
  destruct (h hA) as [hB | hC].
  - exact hB.
  - exfalso. apply hnc. exact hC.
Qed.

```

Arbre de dérivation :

$[hA:A] \quad h:A \rightarrow B \vee C$

$$\begin{array}{c}
\frac{}{\vdash B \vee C} \quad (\rightarrow E) \\
\\
\frac{[hB:B]}{\vdash B} \quad (Ax) \qquad \frac{[hC:C] \quad hnc:\neg C}{\vdash \text{False}} \quad (\rightarrow E) \\
\qquad \qquad \qquad \frac{}{\vdash \text{False}} \quad (\text{ex falso}) \\
\qquad \qquad \qquad \vdash B \\
\hline
\vdash B \quad (\vee E \text{ sur } B \vee C) \\
\\
\frac{h, hnc, hA \vdash B}{\vdash B} \quad (\rightarrow I \text{ sur } hA) \\
\frac{h, hnc \vdash A \rightarrow B}{\vdash A \rightarrow B} \quad (\rightarrow I \text{ sur } hnc) \\
\frac{h \vdash \neg C \rightarrow A \rightarrow B}{\vdash \neg C \rightarrow A \rightarrow B} \quad (\rightarrow I \text{ sur } h) \\
\vdash (A \rightarrow B \vee C) \rightarrow \neg C \rightarrow A \rightarrow B
\end{array}$$

Exercice 2 — Arbres d'expressions arithmétiques `expr` (15 pts)

Inductive `expr : Set := Lit (n : nat) | Add (e1 e2 : expr) | Mul (e1 e2 : expr).`

1. Représentation graphique de `exple_8` (1 pt)

Definition `exple_8 := Add (Lit 5) (Mul (Lit 2) (Lit 6)).`

```

      Add
     /  \
  Lit 5  Mul
       /  \
    Lit 2  Lit 6

```

2. Évaluation de `eval exple_8` (2 pts)

```

Fixpoint eval (e : expr) : nat :=
  match e with
  | Lit n      => n
  | Add e1 e2 => eval e1 + eval e2
  | Mul e1 e2 => eval e1 * eval e2
  end.

```

```

eval exple_8
= eval (Lit 5) + eval (Mul (Lit 2) (Lit 6))   [eA]
= 5 + eval (Mul (Lit 2) (Lit 6))              [eL]
= 5 + (eval (Lit 2) * eval (Lit 6))           [eM]
= 5 + (2 * 6)                                 [eL, eL]

```

```
= 5 + 12
= 17
```

Résultat : 17

3. Évaluation de `size_exple_8` (2 pts)

```
Fixpoint size (e : expr) : nat :=
  match e with
  | Lit _      => 1                (* sL *)
  | Add e1 e2 => size e1 + size e2 (* sA *)
  | Mul e1 e2 => size e1 + size e2 (* sM *)
  end.
```

```
size_exple_8
= size (Lit 5) + size (Mul (Lit 2) (Lit 6)) [sA]
= 1 + size (Mul (Lit 2) (Lit 6))           [sL]
= 1 + (size (Lit 2) + size (Lit 6))         [sM]
= 1 + (1 + 1)                               [sL, sL]
= 1 + 2
= 3
```

Résultat : 3

4. Preuve papier de `size_doubleLits` (4 pts)

```
Fixpoint doubleLits (e : expr) : expr :=
  match e with
  | Lit n      => Lit (2 * n)        (* dL *)
  | Add e1 e2 => Add (doubleLits e1) (doubleLits e2) (* dA *)
  | Mul e1 e2 => Mul (doubleLits e1) (doubleLits e2) (* dM *)
  end.
```

Lemme : $\forall e, \text{size}(\text{doubleLits } e) = \text{size } e$ — par induction sur e .

Cas Lit n :

```
size (doubleLits (Lit n))
= size (Lit (2*n)) [dL]
= 1                [sL]
= size (Lit n)     [sL] ✓
```

Cas Add $e1$ $e2$, IH1 : $\text{size}(\text{doubleLits } e1) = \text{size } e1$, IH2 : $\text{size}(\text{doubleLits } e2) = \text{size } e2$:

```
size (doubleLits (Add e1 e2))
= size (Add (doubleLits e1) (doubleLits e2)) [dA]
= size (doubleLits e1) + size (doubleLits e2) [sA]
= size e1 + size e2                           [IH1, IH2]
= size (Add e1 e2)                            [sA] ✓
```

Cas Mul e1 e2, IH1, IH2 :

```

size (doubleLits (Mul e1 e2))
= size (Mul (doubleLits e1) (doubleLits e2))    [dM]
= size (doubleLits e1) + size (doubleLits e2)    [sM]
= size e1 + size e2                              [IH1, IH2]
= size (Mul e1 e2)                              [sM] ✓

```

Remarque : la preuve fonctionne de façon identique pour Add et Mul car size ignore les valeurs numériques portées par les littéraux — seule la forme de l'arbre compte. C'est pourquoi doubleLits, qui ne change que les valeurs et pas la structure, préserve size. (Une propriété analogue sur eval, par exemple eval (doubleLits e) = 2 * eval e, serait fausse dès qu'un nœud Mul apparaîtrait, car $(2a) \cdot (2b) = 4ab \neq 2 \cdot (ab)$ en général.)

5. Preuve papier de iter_eval (4 pts)

```

Fixpoint iter {B} (fl : nat -> B) (fa fm : B -> B -> B) (e : expr) : B :=
  match e with
  | Lit n      => fl n
  | Add e1 e2 => fa (iter fl fa fm e1) (iter fl fa fm e2)
  | Mul e1 e2 => fm (iter fl fa fm e1) (iter fl fa fm e2)
  end.

```

Lemme : iter (fun n => n) (fun x y => x+y) (fun x y => x*y) e = eval e — par induction sur e.

Cas Lit n :

```

iter (fun n=>n) (fun x y=>x+y) (fun x y=>x*y) (Lit n)
= (fun n => n) n          [définition de iter, cas Lit]
= n
= eval (Lit n)           [eL] ✓

```

Cas Add e1 e2, IH1, IH2 :

```

iter ... (Add e1 e2)
= (fun x y => x+y) (iter ... e1) (iter ... e2)    [cas Add]
= iter ... e1 + iter ... e2
= eval e1 + eval e2                              [IH1, IH2]
= eval (Add e1 e2)                              [eA] ✓

```

Cas Mul e1 e2, IH1, IH2 :

```

iter ... (Mul e1 e2)
= (fun x y => x*y) (iter ... e1) (iter ... e2)    [cas Mul]
= iter ... e1 * iter ... e2
= eval e1 * eval e2                              [IH1, IH2]
= eval (Mul e1 e2)                              [eM] ✓

```

6. Définition de countMuls avec iter (2 pts)

```
Definition countMuls (e : expr) : nat :=
  iter (fun _ => 0)
    (fun x y => x + y)
    (fun x y => x + y + 1)
    e.
```

Exercice 3 — Prédicat inductif ecart (13 pts)

```
Inductive ecart : nat -> nat -> Prop :=
| en (n : nat) : ecart n n
| eS1 (n m : nat) : ecart n m -> ecart n (S (S m))
| eS2 (n m : nat) : ecart n m -> ecart (S (S n)) m.
```

1. Règles d'inférence (1 pt)

$\frac{}{\text{ecart } n \ n} \text{ (en)}$	$\frac{\Gamma \vdash \text{ecart } n \ m}{\text{ecart } n \ (S \ (S \ m))} \text{ (eS1)}$	$\frac{\Gamma \vdash \text{ecart } n \ m}{\text{ecart } (S \ (S \ n)) \ m} \text{ (eS2)}$
---	---	---

2. Preuves de ecart37 et ecart73 (4 pts)

a) ecart 3 7 Arbre de dérivation :

```
_____ (en)
ecart 3 3
_____ (eS1)
ecart 3 5
_____ (eS1)
ecart 3 7
```

En Rocq :

```
Lemma ecart37 : ecart 3 7.
Proof. apply eS1. apply eS1. apply en. Qed.
```

b) ecart 7 3 Arbre de dérivation :

```
_____ (en)
ecart 3 3
_____ (eS2)
ecart 5 3
_____ (eS2)
ecart 7 3
```

En Rocq :

```

Lemma ecart73 : ecart 7 3.
Proof. apply eS2. apply eS2. apply en. Qed.

```

3. ecart_diff — induction sur k (4 pts)

```

Lemma ecart_diff (n m k : nat) : m = n + 2 * k -> ecart n m.
Proof.
  induction k as [| k IH] in m |- *.
  - (* k = 0
      Hypothèses : n m : nat, H : m = n + 2*0 i.e. m = n
      But : ecart n m *)
    intros H.
    assert (m = n) by lia.
    subst. apply en.
  - (* k = S k', IH : ∀m, m = n + 2*k -> ecart n m
      Hypothèses : H : m = n + 2*(S k)
      But : ecart n m *)
    intros H.
    apply eS1.
    apply (IH (n + 2 * k)).
    reflexivity.
Qed.

```

Idée du cas S k : on pose $m_0 := n + 2*k$. Comme $m = n + 2*(S\ k) = m_0 + 2 = S(S\ m_0)$, il suffit de montrer $\text{ecart}\ n\ m_0$ (par IH, appliqué avec son hypothèse triviale $m_0 = n + 2*k$), puis d'appliquer $eS1$ pour obtenir $\text{ecart}\ n\ (S(S\ m_0)) = \text{ecart}\ n\ m$. Cette approche, contrairement à `proche_diff`, n'a pas besoin de distinguer les cas impossibles : la construction est directe.

4. ecart_exists — les trois cas (4 pts)

```

Lemma ecart_exists (n m : nat) :
  ecart n m -> (exists k, m = n + 2 * k) \/ (exists k, n = m + 2 * k).
Proof.
  intros H.
  induction H as [p | p q Hpq IHpq | p q Hpq IHpq].
  - (* en p : ecart p p *)
    left. exists 0. lia.
  - (* eS1 p q : ecart p q -> ecart p (S(S q)) *)
    destruct IHpq as [[k Hk] | [k Hk]].
    + (* Hk : q = p + 2*k *)
      left. exists (S k). lia.
    + (* Hk : p = q + 2*k *)
      destruct k as [| k'].

```

```

    * (* k = 0 : p = q *)
      left. exists 1. lia.
    * (* k = S k' : p = q + 2*(S k') *)
      right. exists k'. lia.

- (* eS2 p q : ecart p q -> ecart (S(S p)) q *)
  destruct IHpq as [[k Hk] | [k Hk]].
+ (* Hk : q = p + 2*k *)
  destruct k as [| k'].
  * (* k = 0 : q = p *)
    right. exists 1. lia.
  * (* k = S k' : q = p + 2*(S k') *)
    left. exists k'. lia.
+ (* Hk : p = q + 2*k *)
  right. exists (S k). lia.
Qed.

```

Exercice 4 — listes, prédicat universel, réflexion (20 pts)

```

Fixpoint forallb {A} (p : A -> bool) (l : list A) : bool :=
  match l with
  | []      => true
  | h :: t => p h && forallb p t
  end.

Inductive All {A} (p : A -> Prop) : list A -> Prop :=
  | All_nil                : All p []
  | All_cons (h : A) (t : list A) : p h -> All p t -> All p (h :: t).

```

On pose $p := \text{fun } n \Rightarrow 3 \leq n$.

1. Évaluation de $\text{forallb } p \text{ [5; 10; 3; 7]}$ (2 pts)

```

forallb p [5; 10; 3; 7]
= p 5 && forallb p [10; 3; 7]
= true && forallb p [10; 3; 7]           [5 ≥ 3]
= forallb p [10; 3; 7]
= p 10 && forallb p [3; 7]
= true && forallb p [3; 7]               [10 ≥ 3]
= forallb p [3; 7]
= p 3 && forallb p [7]
= true && forallb p [7]                  [3 ≥ 3]
= forallb p [7]
= p 7 && forallb p []
= true && forallb p []                   [7 ≥ 3]
= forallb p []

```

= true

Résultat : true

2. Preuve de All (fun n => 3 <= n) [5; 10; 3; 7] (2 pts)

```
apply All_cons. lia. (* 3 <= 5 *)
apply All_cons. lia. (* 3 <= 10 *)
apply All_cons. lia. (* 3 <= 3 *)
apply All_cons. lia. (* 3 <= 7 *)
apply All_nil.
```

3. Preuve de forallb_All (5 pts)

```
Lemma forallb_All {A} (p : A -> bool) (l : list A) :
  forallb p l = true -> All (fun x => p x = true) l.
Proof.
  induction l as [| h t IH].
  - (* l = [] *)
    intros _. apply All_nil.
  - (* l = h :: t, IH : forallb p t = true -> All (fun x => p x = true) t *)
    intros H. simpl in H.
    destruct (p h) eqn:E.
    + (* p h = true *)
      simpl in H.
      apply All_cons.
      * exact E.
      * apply IH. exact H.
    + (* p h = false : contradiction car forallb p (h::t) = false && ... = false ≠ true *)
      simpl in H. discriminate H.
Qed.
```

4. Preuve de All_forallb (5 pts)

```
Lemma All_forallb {A} (p : A -> bool) (l : list A) :
  All (fun x => p x = true) l -> forallb p l = true.
Proof.
  intros H.
  induction H as [| h t Hph Hpt IH].
  - (* All_nil *)
    simpl. reflexivity.
  - (* All_cons h t Hph Hpt IH : Hph : p h = true, Hpt : All p t, IH : forallb p t = true *)
    simpl.
    rewrite Hph.
```

```
simpl.
exact IH.
Qed.
```

5. Preuve de All_app (équivalence) (6 pts)

```
Lemma All_app {A} (p : A -> Prop) (l1 l2 : list A) :
  All p (l1 ++ l2) <-> All p l1 /\ All p l2.
```

Proof.

```
induction l1 as [| h t IH].
```

```
- (* l1 = [] *)
  simpl. split.
  + intros H. split.
    * apply All_nil.
    * exact H.
  + intros [_ H2]. exact H2.

- (* l1 = h :: t, IH : All p (t ++ l2) <-> All p t /\ All p l2 *)
  simpl. split.
  + intros H.
    inversion H as [| h' t' Hph HAll Heq1 Heq2].
    destruct (proj1 IH HAll) as [Ht Hl2].
    split.
    * apply All_cons. exact Hph. exact Ht.
    * exact Hl2.
  + intros [H1 H2].
    inversion H1 as [| h' t' Hph Ht Heq1 Heq2].
    apply All_cons.
    * exact Hph.
    * apply (proj2 IH). split. exact Ht. exact H2.
```

Qed.

Détail du sens direct (cas $h :: t$) : de $\text{All } p ((h::t)++l2) = \text{All } p (h::(t++l2))$, on extrait par inversion $Hph : p \ h$ et $HAll : \text{All } p (t++l2)$. L'hypothèse d'induction IH appliquée à $HAll$ donne $\text{All } p \ t \ /\ \text{All } p \ l2$. On reconstruit alors $\text{All } p (h::t)$ avec $\text{All_cons } Hph \ Ht$, et on conserve $\text{All } p \ l2$ tel quel.

Détail du sens réciproque : de $H1 : \text{All } p (h::t)$, on extrait $Hph : p \ h$ et $Ht : \text{All } p \ t$. L'IH (sens réciproque) appliquée à $(Ht, H2)$ donne $\text{All } p (t++l2)$, et $\text{All_cons } Hph$ permet de conclure $\text{All } p (h :: (t++l2)) = \text{All } p ((h::t)++l2)$.